Laboratorio de Control Térmico: TempLABUdeA

Departamento de Ingeniería Electrónica
Universidad de Antioquia

24 de marzo de 2026

Introducción

El propósito de este laboratorio es diseñar, implementar y analizar un sistema de control de temperatura basado en el TempLABUdeA, tomando como referencia el LABORATORIO 1. El sistema consta de dos elementos calefactores, dos sensores de temperatura y una interfaz de microcontrolador. Los estudiantes aplicarán técnicas de modelado físico, síntesis de control PID y validación experimental.

Objetivos de la Práctica

1. Desarrollar el modelo dinámico continuo de primer orden para cada zona térmica, considerando conducción, convección y radiación.
2. Diseñar y sintonizar controladores PID independientes para cada secuencia de temperatura, asegurando estabilidad y desempeño especificado.
3. Implementar el controlador en el sistema TempLABUdeA y validar el comportamiento experimental frente al modelo teórico.

Procedimiento

Ejemplo: Cuantificación del offset con controlador P-only

Objetivo Cuantificar el desplazamiento (offset) entre la consigna T_{SP} y la temperatura medida T_{PV} al usar un controlador proporcional-solo (P-only).

Control proporcional-solo El controlador P-only ajusta la señal de control $Q(t)$ en función del error $e(t) = T_{SP} - T_{PV}(t)$:

$$Q(t) = Q_{\text{bias}} + K_c\big(T_{SP} - T_{PV}(t)\big) \tag{1}$$
$$= Q_{\text{bias}} + K_c\, e(t). \tag{2}$$

Aquí,

- Q_{bias} es un valor constante (normalmente el valor de Q en modo manual al pasar a automático) que evita saltos al activarse el controlador $Q_{\text{bias}} = 0$[1]
- K_c es la ganancia proporcional.

Procesos no integradores y offset permanente Dado que el sistema TempLABUdeA regresa a la temperatura ambiente cuando $Q = 0$, es un proceso no integrador y un control P-only deja un error en estado estacionario:

$$e_\infty = T_{SP} - T_{PV,\infty}. \tag{3}$$

La práctica consiste en calcular y verificar este offset experimental.

Sintonía P-only (ITAE) Se emplea la correlación ITAE para seguimiento de consigna, usando parámetros FOPDT (K_p, τ_p, θ_p):

$$K_c = 0{,}20 \left(\frac{\tau_p}{\theta_p}\right)^{1{,}22} \quad \text{(seguimiento de set-point)}. \tag{4}$$

Con este K_c y conocido Q_{bias}, ejecutar la prueba y medir e_∞.

2. Diseño de controlador PID mediante Ziegler–Nichols

1. Configure el sistema en lazo cerrado con acción proporcional pura.
2. Aumente la ganancia K_c hasta que aparezcan oscilaciones sostenidas. Anote:

$$K_u = \text{ganancia crítica}, \quad P_u = \text{periodo de oscilación}.$$

3. Estime los parámetros del PID con la regla de Ziegler–Nichols para control PID:

$$K_p = 0{,}6\, K_u, \quad T_i = 0{,}5\, P_u, \quad T_d = 0{,}125\, P_u.$$

Tutorial corto: Método de Ziegler–Nichols

1. Ponga $I = D = 0$ y aumente K_p hasta que la salida oscile con amplitud constante.
2. Mida el valor de K_u y el periodo P_u de dichas oscilaciones.
3. Calcule:

$$K_p = 0{,}6\, K_u, \tag{5}$$
$$T_i = 0{,}5\, P_u, \tag{6}$$
$$T_d = 0{,}125\, P_u. \tag{7}$$

4. Implemente el controlador completo:

$$u(t) = K_p\Big[e(t) + \frac{1}{T_i}\int_0^t e(\tau)\, d\tau + T_d\, \frac{de}{dt}\Big].$$

[1] En TempLABUdeA los calentadores siempre están inicializados en modo manual con $Q = 0$ antes de activar el controlador. Por definición, Q_{bias} es el valor manual de la señal de control justo antes de pasar a automático, y dado que en este sistema dicho valor es 0 %, se toma $Q_{\text{bias}} = 0$.

3. Implementación práctica

1. Cargue el firmware base en la placa Arduino y abra la interfaz en Python.
2. Desarrolle el script que:
   - Lee $T_1(t)$ y $T_2(t)$.
   - Aplica las señales de control $Q_1(t)$ y $Q_2(t)$ con el PID diseñado.
   - Registra en archivo los datos de tiempo, temperatura y potencia.
3. Realice pruebas de escalón y distintas consigas, grabando siempre la respuesta transitoria y estacionaria.

4. Validación y Análisis

1. Compare la respuesta experimental con la simulada (errores de modelo y de parámetros).
2. Evalúe indicadores de desempeño:
   - Sobreimpulso (%).
   - Tiempo de asentamiento.
   - Error en estado estacionario.
3. Discuta posibles discrepancias y proponga ajustes al modelo o al controlador.

Resultados de Aprendizaje

Al finalizar la práctica, los estudiantes serán capaces de:

1. Aplicar el ***método de Lagrange con disipación*** para modelar sistemas térmicos de primer orden y derivar sus funciones de transferencia.
2. Diseñar, simular e implementar un controlador PID en un sistema real, evaluando su desempeño a través de indicadores de control clásico (sobreimpulso, tiempo de asentamiento, margen de estabilidad).

Entregables

- Documento técnico con:
  - Derivación detallada del modelo dinámico y funciones de transferencia.
  - Cálculo y justificación de los parámetros PID.
  - Comparación gráfica y cuantitativa entre simulación y experimento.
- Código fuente del controlador (Python/C++), incluyendo scripts de adquisición y gráficos.
- Registro de datos crudos (archivo CSV o TXT) y gráficos finales.

Rúbrica de Evaluación

Criterio	0	1	2	3	4	5
Precisión del modelo	Ausente	Erróneo	Parcial	Funcional básico	Bueno	Excelente
Sintonía PID	Ausente	Incorrecta	Subóptima	Aceptable	Buena	Óptima
Comparación simulación/experimento	N/D	>25 %	20–25 %	15–20 %	10–15 %	<10 %
Calidad de informe y código	Ausente	Muy deficiente	Deficiente	Aceptable	Buena	Excelente

Cuadro 1: Rúbrica de evaluación con calificación de 0 a 5.

Consejos de implementación de controladores P-only y PID

A continuación se presentan recomendaciones prácticas basadas en el siguiente esquema de código Python:

```
# Parametros de control
Kc       = 1/0.9
tauI     = 175.0
tauD     = 5.0
Q_bias   = 0.0
ierr     = 0.0
prev_err= 0.0

for i in range(n):
    # Leer temperatura actual
    T1[i] = lab.T1

    # Calculo del error
    err = SP1[i] - T1[i]

    # Termino integral
    ierr += err

    # Termino derivativo
    deriv = err - prev_err
    prev_err = err

    # Accion de control PID
    Q1[i] = Q_bias + Kc*err + (Kc/tauI)*ierr + Kc*tauD*deriv

    # Anti-windup
    if Q1[i] >= 100:
        Q1[i]  = 100
        ierr   -= err
    elif Q1[i] <= 0:
        Q1[i]  = 0
        ierr   -= err
```

```
lab.Q1(Q1[i])
time.sleep(1)
```

- **Inicialización:**
  - Defina siempre un *bias* Q_{bias} (a menudo 0) antes de activar el lazo automático.
  - Inicialice la integral $ierr = 0$ para evitar arrastre de errores anteriores.
- **Período de muestreo constante:**
  - Garantice un muestreo aproximado de $\Delta t = 1\,\text{s}$ (o el valor elegido) usando `time.sleep()`.
  - Calcule el tiempo real transcurrido para posibles ajustes de Δt si el retardo de lectura/ejecución no es despreciable.
- **Antiwindup:**
  - Si la señal $Q(t)$ se satura a sus límites $0 \leq Q \leq 100$, restaure la integral:

$$\text{si } Q = Q_{\text{máx}}, \quad ierr \mathrel{-}= err; \quad \text{si } Q = Q_{\text{mín}}, \quad ierr \mathrel{-}= err.$$

  - Así se evita que la acción integral continúe acumular error cuando el actuador ya está saturado.
- **Selección de parámetros:**
  - Para P-only, omita la parte integral:

$$Q(t) = Q_{\text{bias}} + K_c\, e(t).$$

  - Para PI, compruebe que la constante τ_I esté en las unidades de s.
  - Ajuste fino de K_c y τ_I puede realizarse sobre la marcha comparando la respuesta real con la simulada.
- **Registro y visualización:**
  - Almacene vectores $\{t, Q, T, SP\}$ en un archivo CSV para análisis posterior:

```
data = np.vstack((t,Q,T,SP)).T
```

  - Use `matplotlib` para gráficos en tiempo real y para el informe final:
    - Gráfico de $T(t)$ comparado con $SP(t)$.
    - Gráfico de la señal de control $Q(t)$.
- **Cierre seguro:**
  - Antes de terminar el script, siempre envíe $Q_i = 0$ para apagar los calentadores:

```
lab.Q1(0); lab.Q2(0);
```

  - Cierre la conexión con `lab.close()` dentro de un bloque `try/except` para garantizar la desconexión incluso ante errores o interrupciones.